

Actor Critic Methods

CS 224R

Course reminders

- Start forming final project groups (survey due next Wednesday)
- Homework 1 due next Friday

Recap of Last Time

Online reinforcement learning with policy gradients

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \left(\left(\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right) - b \right)$$

samples from policy policy log likelihood reward to go baseline



Do more of the above average stuff, less of the below average stuff.

Recap of Last Time

attribute of online RL algorithms

Vanilla policy gradients is fully **on-policy**.

—> only one gradient step per batch of data

Definitions.

on-policy: update uses only data from current policy

off-policy: update can reuse data from other, past policies

Importance weights for off-policy policy gradient:

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})}{\pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \left(\left(\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right) - b \right)$$

Note: Not a silver bullet. Only suitable when policies are very similar.

The plan for today

Actor critic methods

1. Improving policy gradients
2. How to estimate the value of a policy
 - a. Sample & directly supervise
 - b. Use your own estimate
 - c. Something in-between?
3. Off-policy actor-critic
 - a. Importance weights & constraining step size
 - b. Full off-policy version with replay buffers

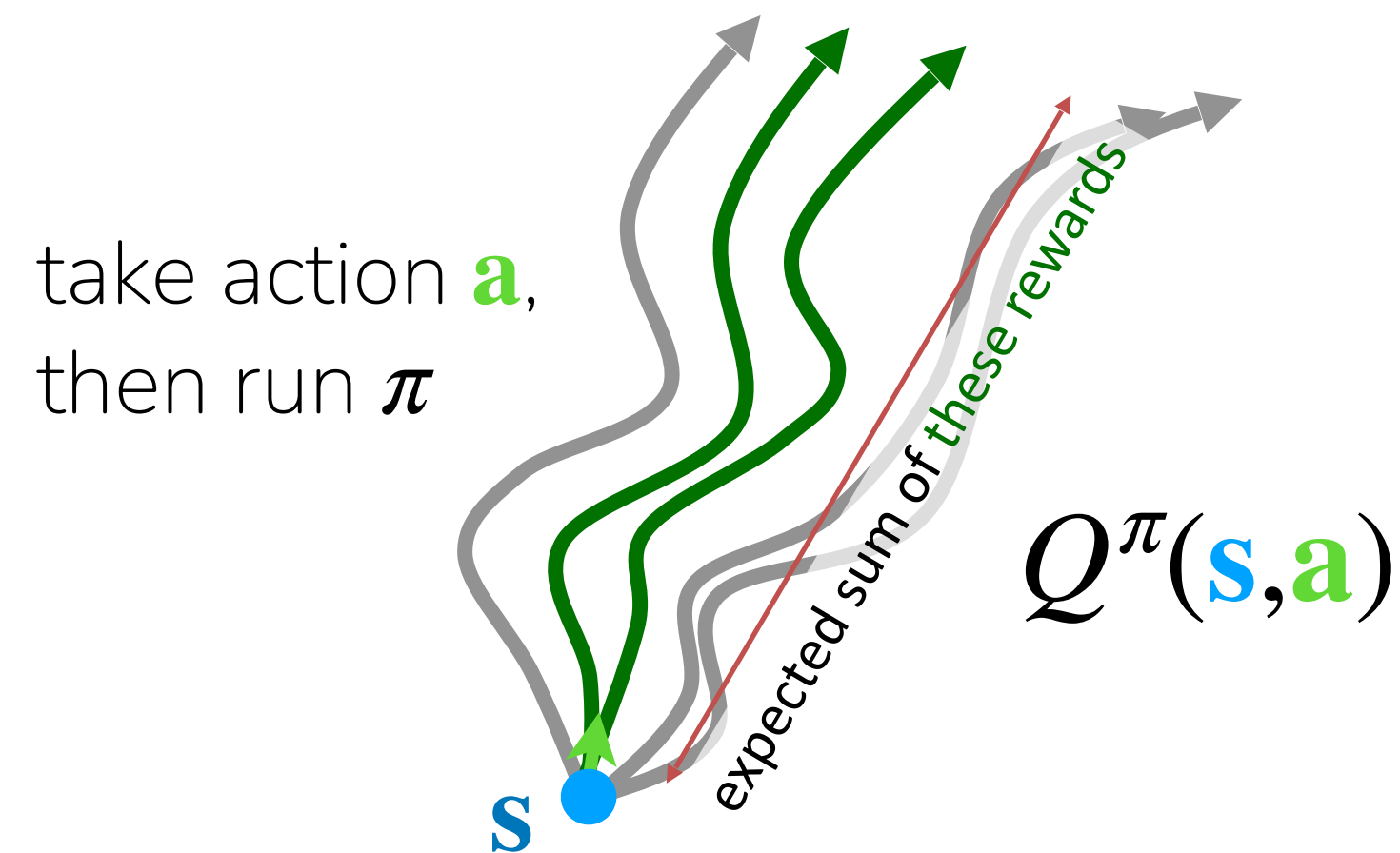
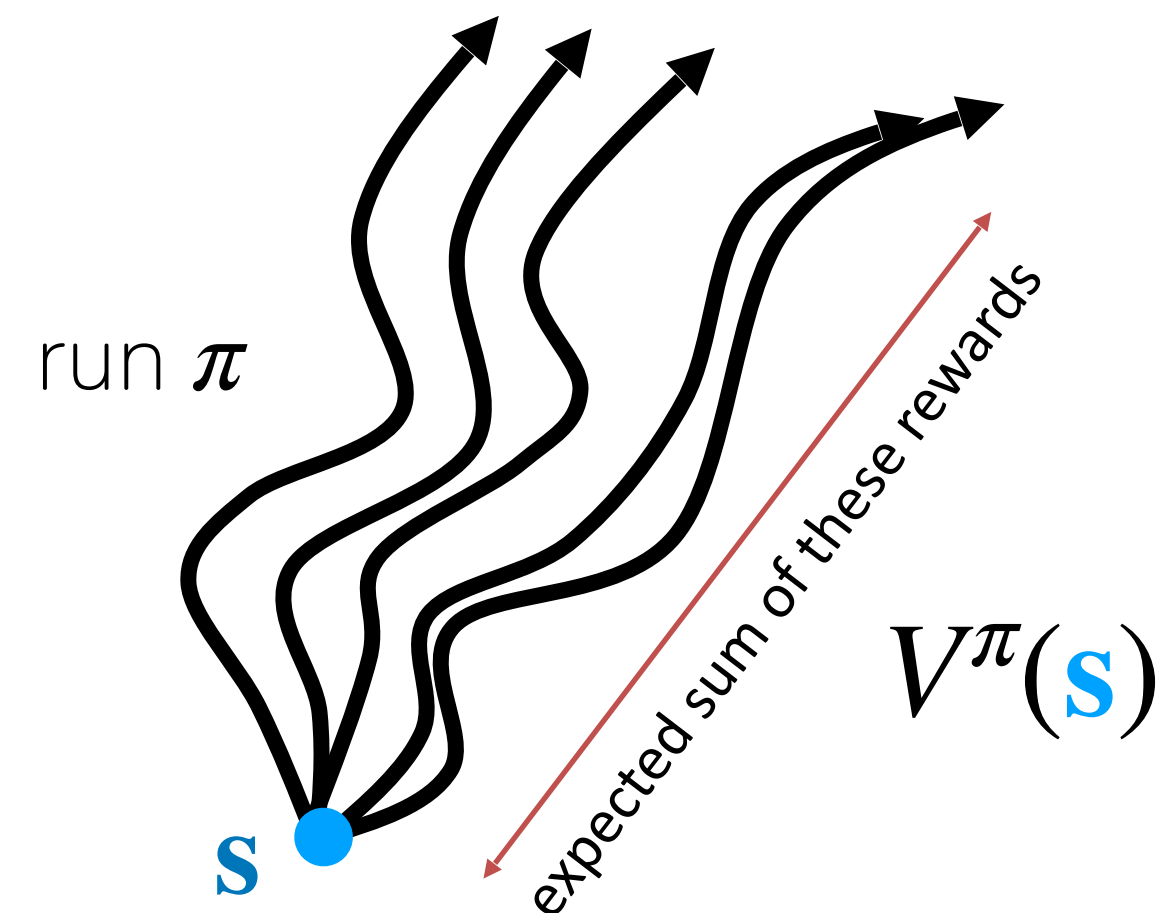
Key learning goals:

- How to estimate how good a state and action is for a policy
- How to use those estimates to form a more efficient RL algorithm

Revisiting Some Useful Objects

value function $V^\pi(\mathbf{s})$ - future expected rewards starting at $\mathbf{s}$ and following π

Q-function $Q^\pi(\mathbf{s}, \mathbf{a})$ - future expected rewards starting at $\mathbf{s}$, taking $\mathbf{a}$, then following π



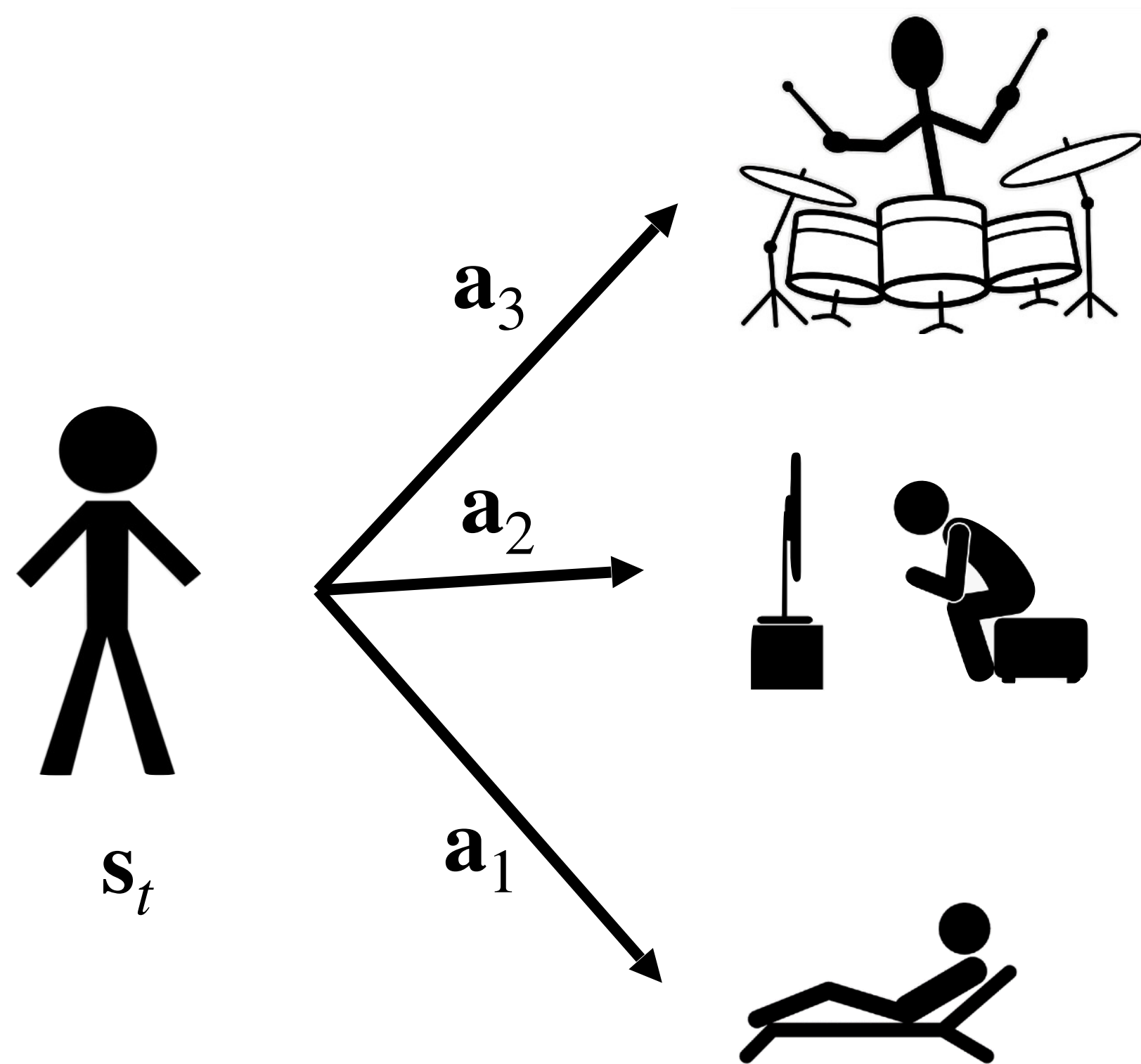
Useful relation: $V^\pi(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi(\cdot | \mathbf{s})} [Q^\pi(\mathbf{s}, \mathbf{a})]$

advantage $A^\pi(\mathbf{s}, \mathbf{a})$ - how much better it is to take $\mathbf{a}$ than to follow policy π at state $\mathbf{s}$

$$A^\pi(\mathbf{s}, \mathbf{a}) = Q^\pi(\mathbf{s}, \mathbf{a}) - V^\pi(\mathbf{s})$$

Let's look at an example

Reward = 1 if I can play it in
a month, 0 otherwise



Question:

For each action, what are these?

Value function: $V^\pi(s_t) = ?$

Q function: $Q^\pi(s_t, a_t) = ?$

Advantage function: $A^\pi(s_t, a_t) = ?$

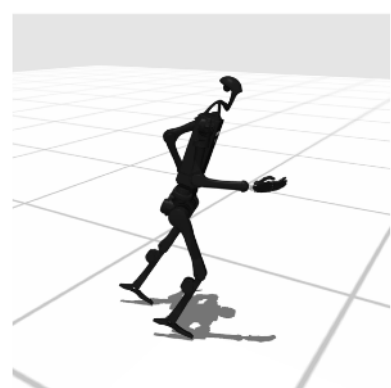
Current policy: $\pi(a_1 | s) = 1 \quad \forall s$

What is dissatisfying about policy gradients?

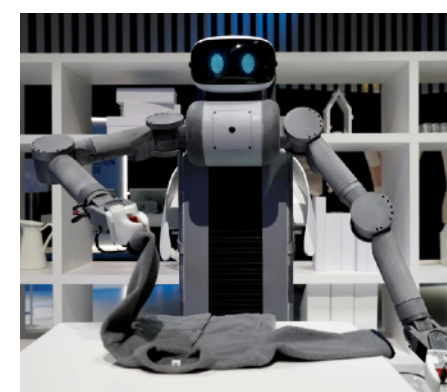


$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \left(\left(\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right) - b \right)$$

samples from policy policy log likelihood reward to go baseline



τ^4 : one small step forward then falls backwards



τ^2 : folds only the sleeves
 τ^3 : flattens the jacket but does not fold it
 τ^4 : folds the jacket

-> pushes down likelihood of step forward

-> with sparse rewards, don't utilize τ^2 or τ^3

Policy gradients doesn't make efficient use of data! 💡 Can we learn what is good & bad?

Improving policy gradients

Estimating reward to go

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \underbrace{\left(\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)}_{\text{"reward to go"}}$$

estimate of future rewards if we take $\mathbf{a}_{i,t}$ in state $\mathbf{s}_{i,t}$

$$\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$$

Can we get a better estimate?

$$\sum_{t'=t}^T E_{\pi_{\theta}} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t] = Q(\mathbf{s}_t, \mathbf{a}_t)$$

true expected rewards to go

This would be way better!

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) Q(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

Improving policy gradients

What about baselines?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) (Q(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) - \cancel{b})$$

$V^{\pi}(\mathbf{s}_t)$

$$b = \text{average reward} = \frac{1}{N} \sum_i Q(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

$$V(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_{\theta}(\cdot | \mathbf{s}_t)} [Q(\mathbf{s}_t, \mathbf{a}_t)]$$

Recall: $A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) - V^{\pi}(\mathbf{s}_t)$

With baseline:

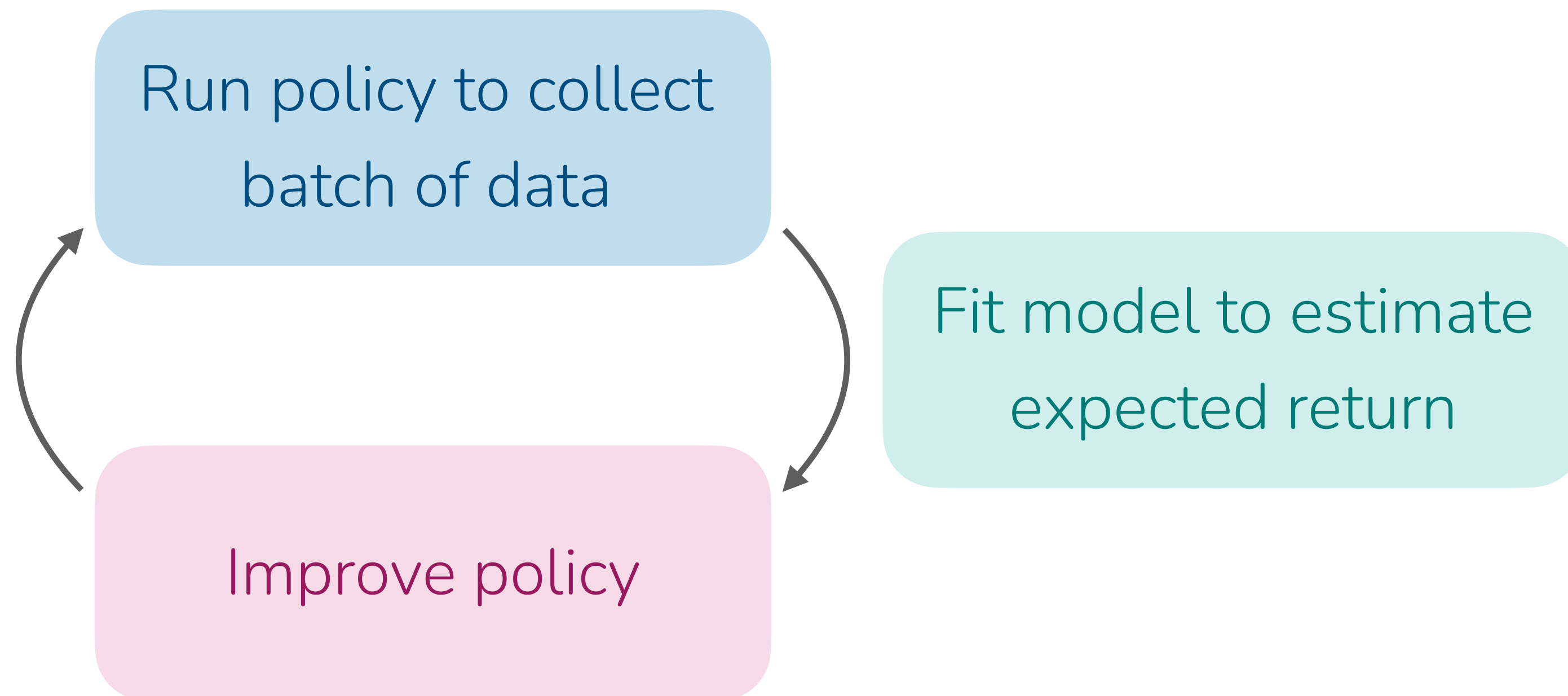
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

Better estimates of A lead to less noisy gradients!

Online RL Outline

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

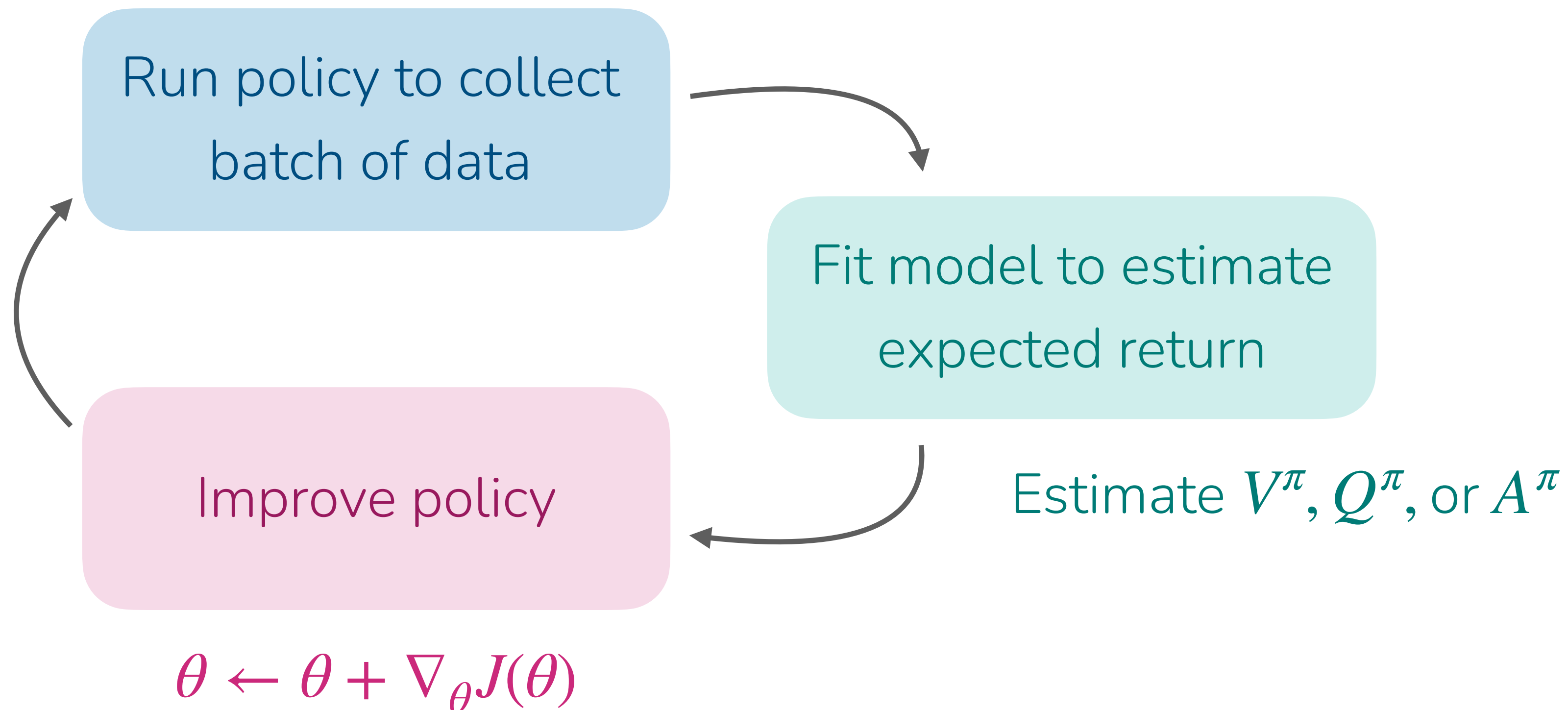
First: Initialize the policy (randomly, with imitation learning, with heuristics)



Online RL Outline

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

First: Initialize the policy (randomly, with imitation learning, with heuristics)



The plan for today

Actor critic methods

1. Improving policy gradients
- 2. How to estimate the value of a policy**
 - a. Sample & directly supervise
 - b. Use your own estimate
 - c. Something in-between?
3. Off-policy actor-critic
 - a. Importance weights & constraining step size
 - b. Full off-policy version with replay buffers

Key learning goals:

- How to estimate how good a state and action is for a policy
- How to use those estimates to form a better RL algorithm

Estimating expected return

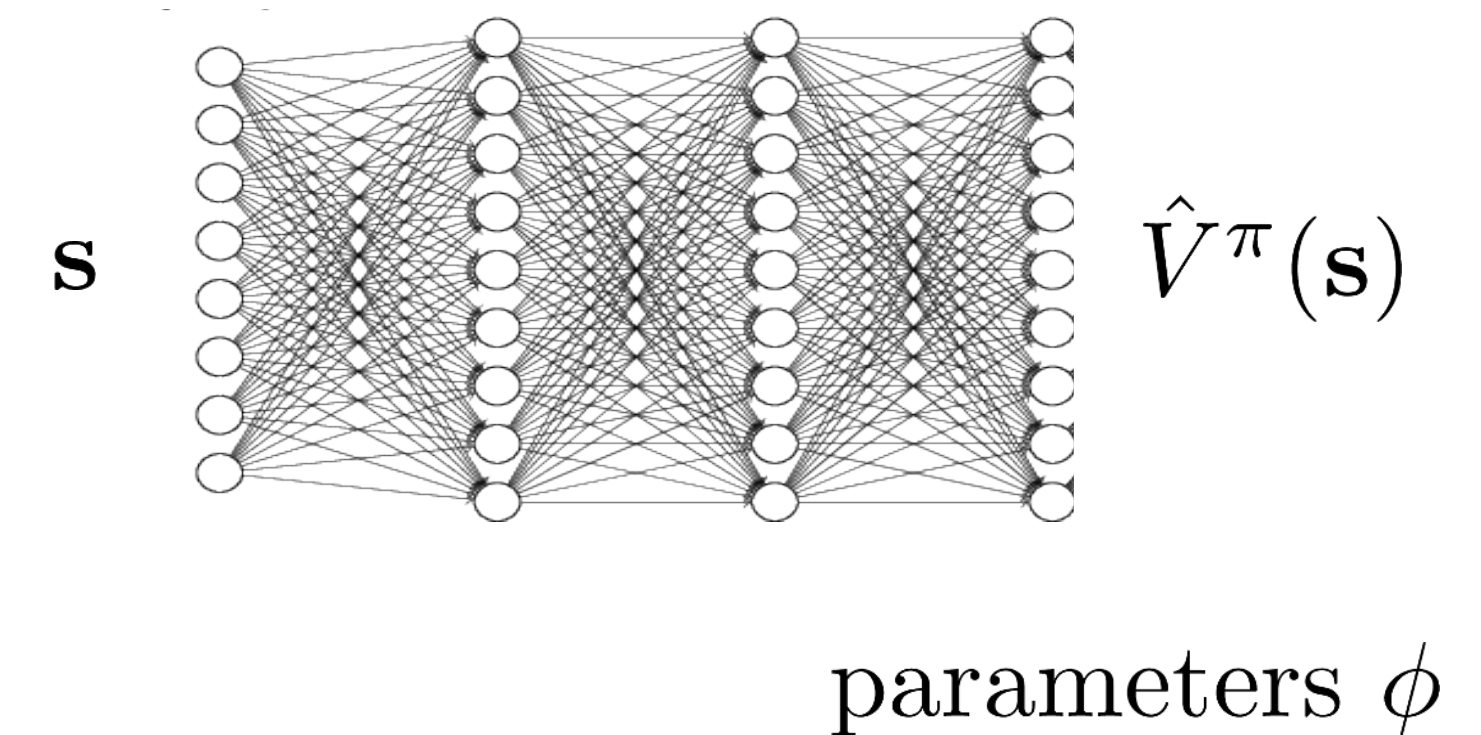
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

Should we fit V^{π} , Q^{π} , or A^{π} ?

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) - V^{\pi}(\mathbf{s}_t)$$

$$\begin{aligned} Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) &= \sum_{t'=t}^T E_{\pi_{\theta}} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t] \\ &= r(\mathbf{s}_t, \mathbf{a}_t) + \sum_{t'=t+1}^T E_{\pi_{\theta}} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t] \\ &= r(\mathbf{s}_t, \mathbf{a}_t) + E_{\mathbf{s}_{t+1} \sim p(\cdot | \mathbf{s}_t, \mathbf{a}_t)} [V^{\pi}(\mathbf{s}_{t+1})] \\ &\approx r(\mathbf{s}_t, \mathbf{a}_t) + V^{\pi}(\mathbf{s}_{t+1}) \quad (\text{use the sampled } \mathbf{s}_{t+1}) \end{aligned}$$

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^{\pi}(\mathbf{s}_{t+1}) - V^{\pi}(\mathbf{s}_t) \quad \text{Let's just fit } V^{\pi}!$$

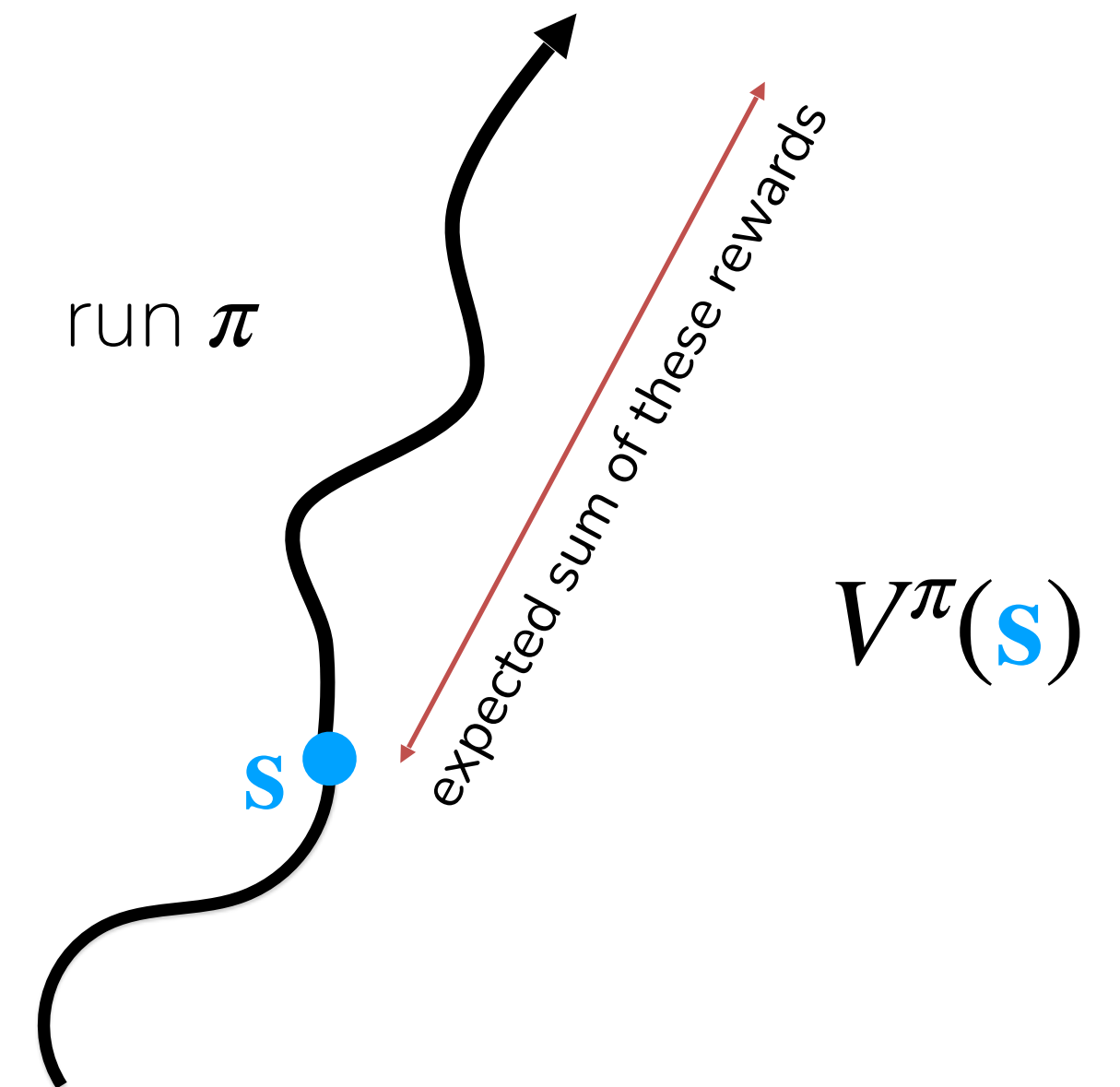


Estimating V^π

Version 1: Monte Carlo estimation

$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$$

<- original single sample estimate



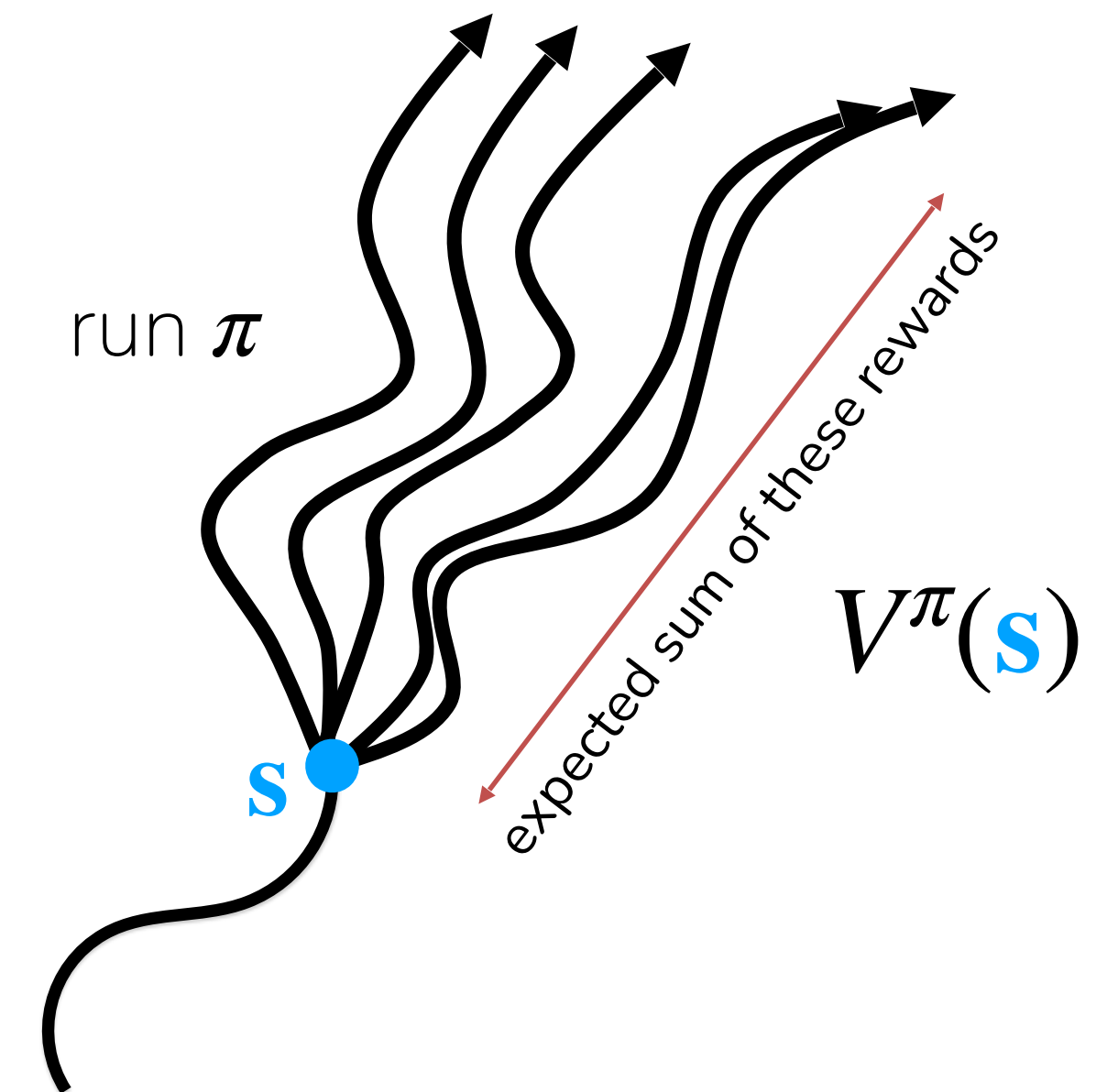
Estimating V^π

Version 1: Monte Carlo estimation

$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \quad \leftarrow \text{original single sample estimate}$$

$$V^\pi(\mathbf{s}_t) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \quad \leftarrow \text{multi-sample estimate}$$

(but can't reset the world)



Estimating V^π

Version 1: Monte Carlo estimation

$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \quad \leftarrow \text{original single sample estimate}$$

$$V^\pi(\mathbf{s}_t) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \quad \leftarrow \text{multi-sample estimate}$$

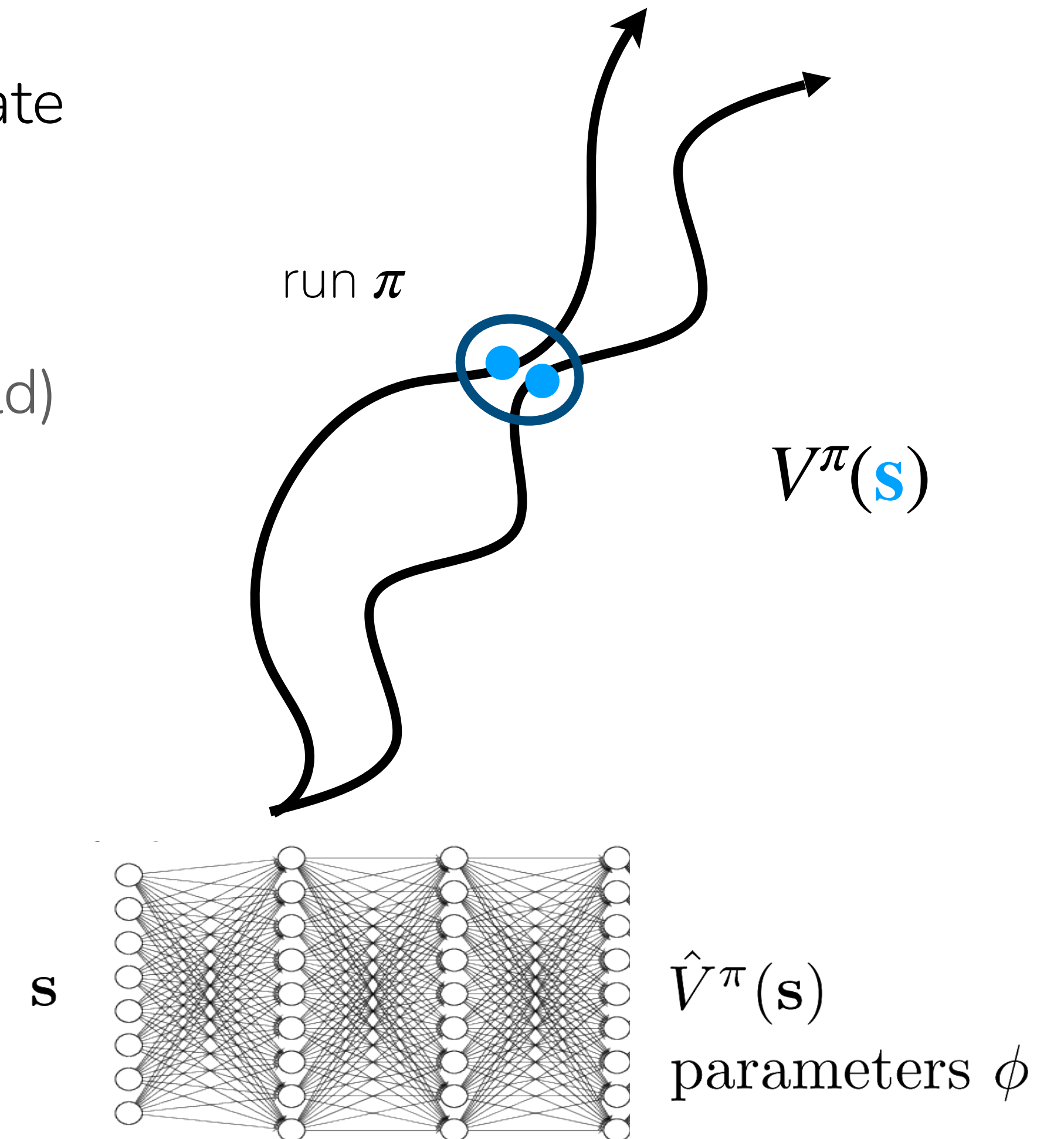
(but can't reset the world)

Step 1: Aggregate dataset of single sample estimates:

$$\left\{ \underbrace{\left(\mathbf{s}_{i,t}, \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)}_{y_{i,t}} \right\}$$

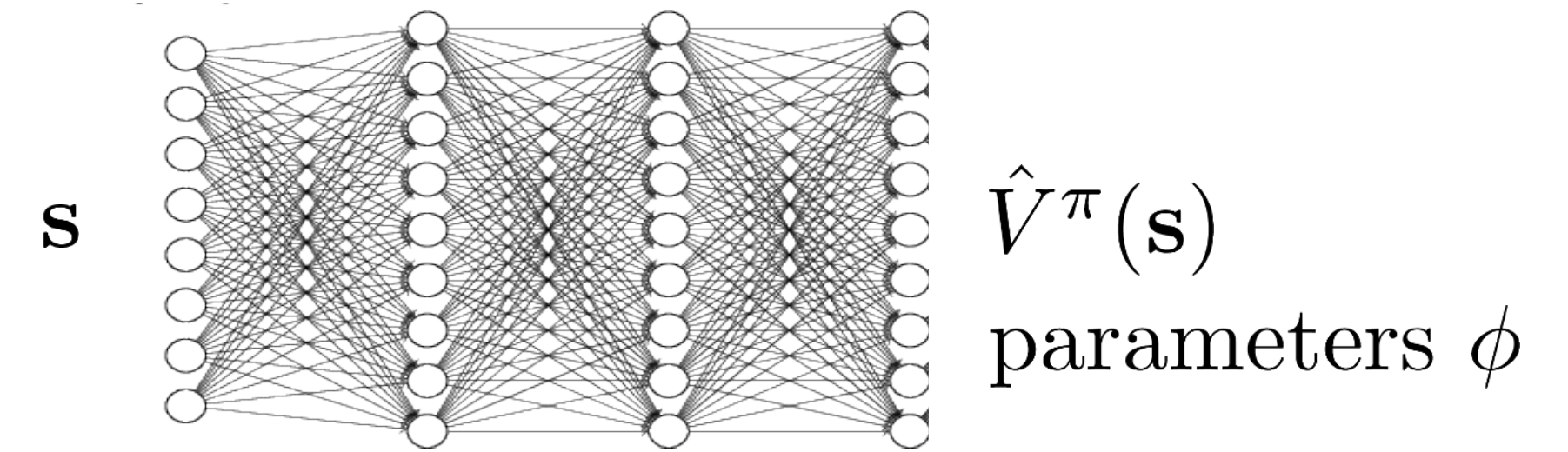
Step 2: Supervised learning to fit estimated value function

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$



Estimating V^π

Version 2: Bootstrapping



Monte Carlo target: $y_{i,t} = \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$

$$\begin{aligned} \text{ideal target: } y_{i,t} &= \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_{i,t}] \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \sum_{t'=t+1}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_{i,t+1}] \\ &\approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \underbrace{V^\pi(\mathbf{s}_{i,t+1})}_{\text{directly use previous fitted value function!}} \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1}) \end{aligned}$$

directly use previous fitted value function!

Step 1: Aggregate dataset of “bootstrapped” estimates:

$$\text{training data: } \left\{ \left(\mathbf{s}_{i,t}, \underbrace{r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1})}_{y_{i,t}} \right) \right\} \quad \leftarrow \text{update labels every gradient update!}$$

Step 2: Supervised learning to fit estimated value function

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

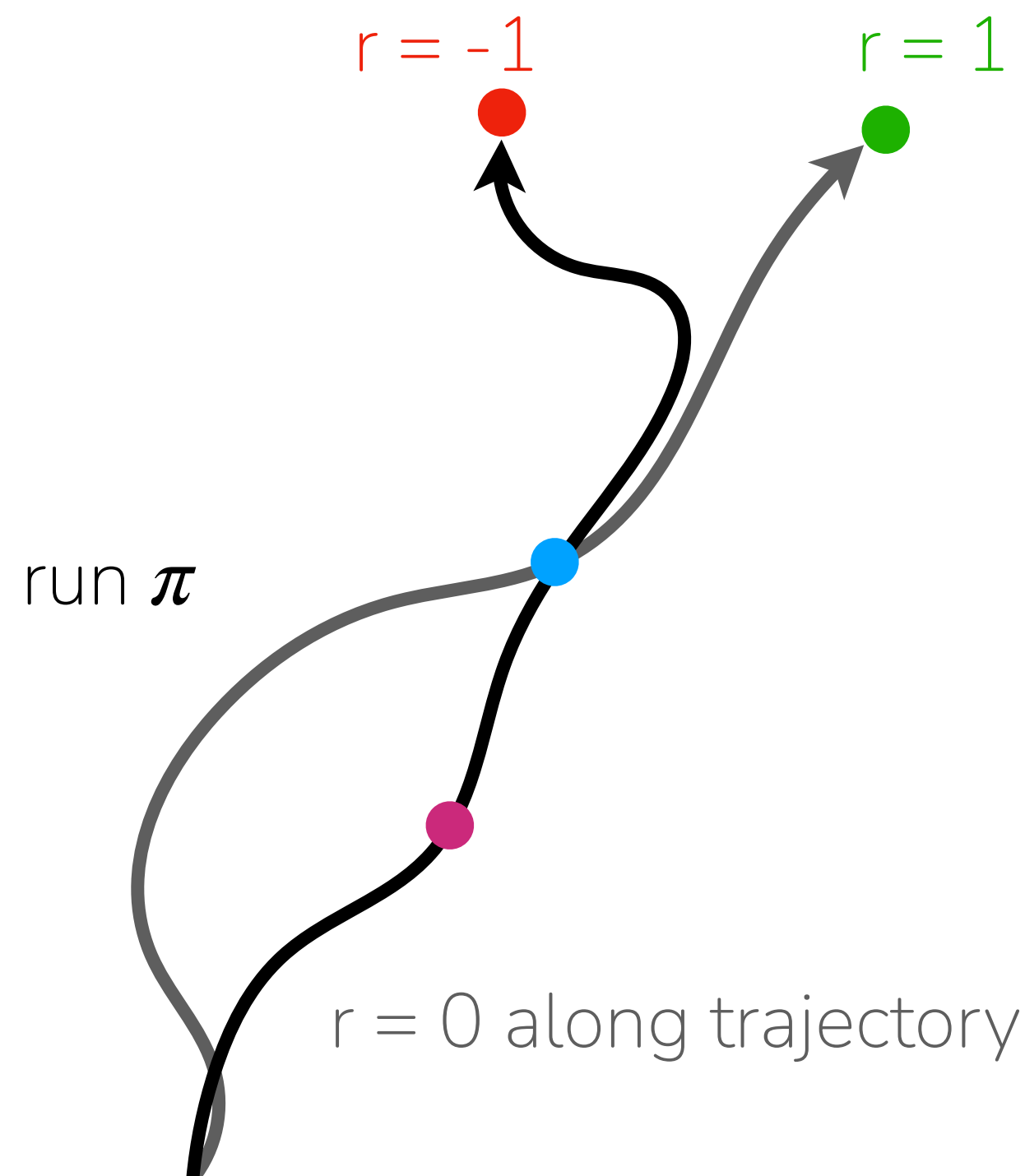
Also referred to as a form of
temporal difference learning

Estimating V^π : Monte Carlo vs. Bootstrap

Monte Carlo $y_{i,t} = \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$ supervise with roll-out's summed rewards

Bootstrapped $y_{i,t} = r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1})$ supervise using reward and current value estimate

Let's look at an example:



Think-pair-share:

$$\hat{V}_{MC}^\pi(\mathbf{s}) \approx ?$$

$$\hat{V}_{MC}^\pi(\mathbf{s}) \approx ?$$

$$\hat{V}_{TD}^\pi(\mathbf{s}) \approx ?$$

$$\hat{V}_{TD}^\pi(\mathbf{s}) \approx ?$$

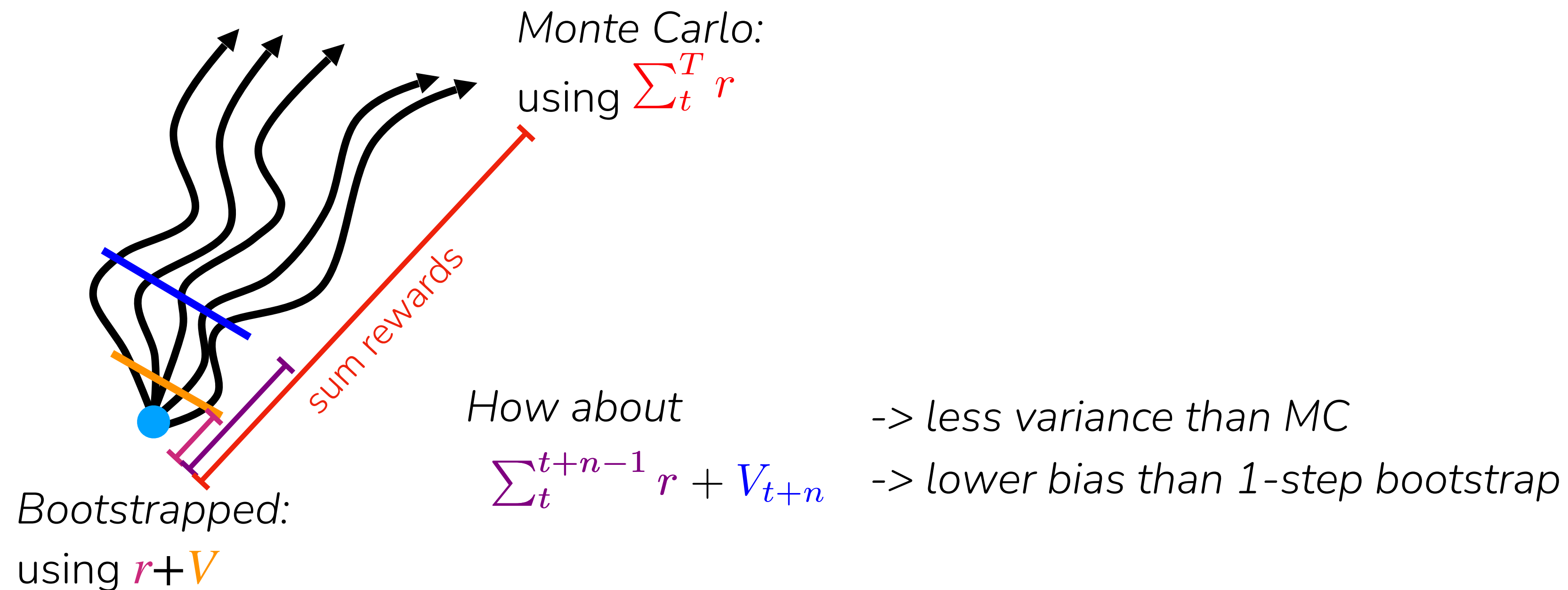
Is there middle ground?
Can we balance bias and variance?

Estimating V^π : Monte Carlo vs. Bootstrap

Monte Carlo $y_{i,t} = \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$ supervise with roll-out's summed rewards

Bootstrapped $y_{i,t} = r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1})$ supervise using reward and current value estimate

N-step returns $y_{i,t} = \sum_{t'=t}^{t+n-1} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+n})$



$n > 1, n < T$ often works the best in practice!

Aside: discount factors

$$y_{i,t} \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_{\phi}^{\pi}(\mathbf{s}_{i,t+1})$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_{\phi}^{\pi}(\mathbf{s}_i) - y_i \right\|^2$$

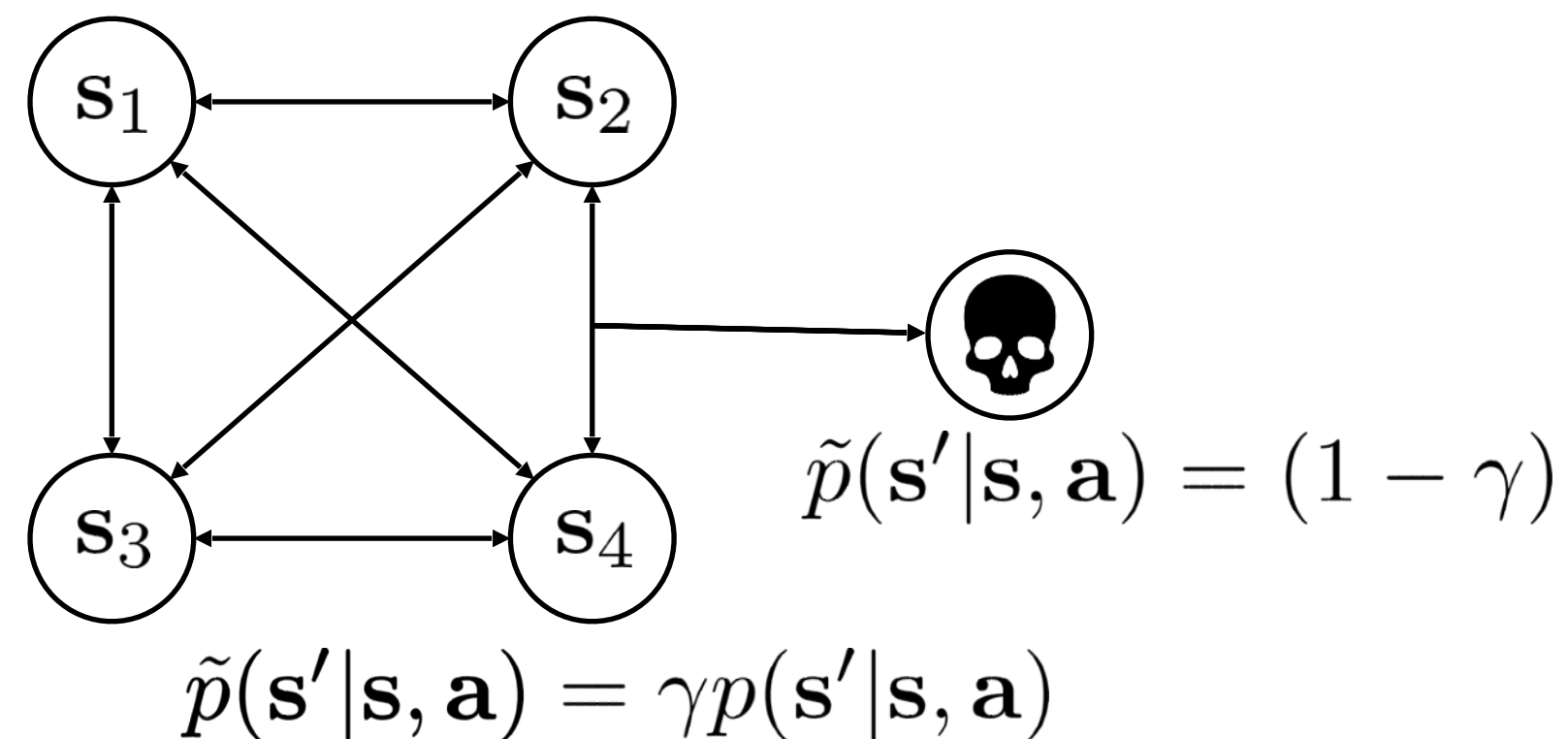
what if T (episode length) is ∞ ?

$\hat{V}_{\phi}^{\pi}$ can get infinitely large in many cases

simple trick: better to get rewards sooner than later γ changes the MDP:

$$y_{i,t} \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{i,t+1})$$

↑
discount factor $\gamma \in [0, 1]$ (0.99 works well)



A Full Algorithm Walkthrough

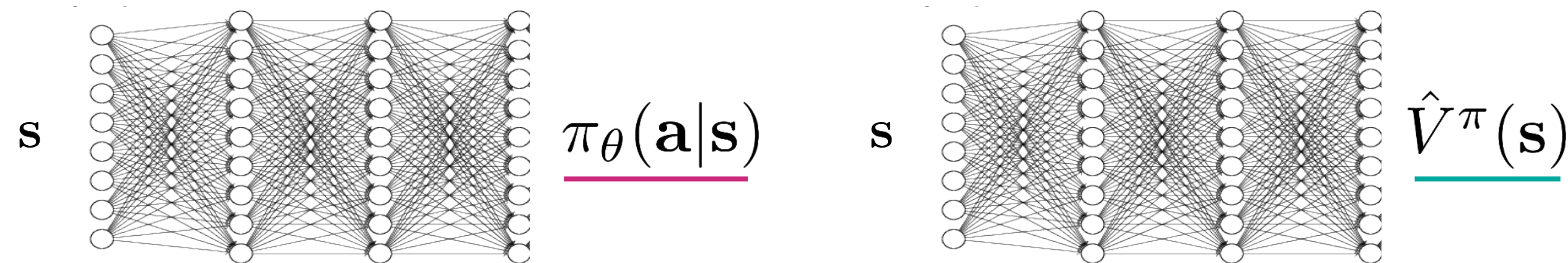
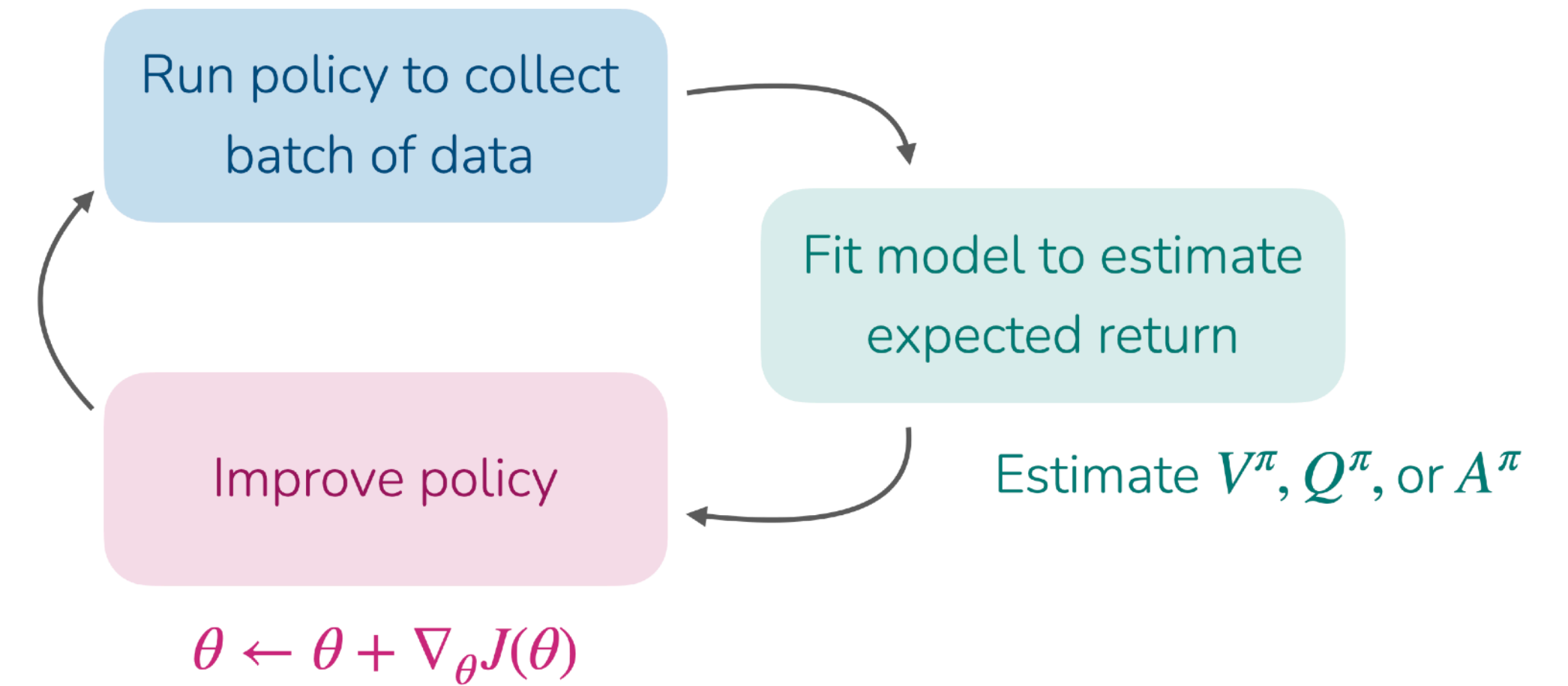
1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_θ

2. Fit $\hat{V}_\phi^{\pi_\theta}$ to summed rewards in data

3. Evaluate $\hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) = r(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \gamma \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t+1,i}) - \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t,i}) \quad \forall t, i$

4. Evaluate $\nabla_\theta J(\theta) \approx \sum_{t,i} \nabla_\theta \log \pi_\theta(\mathbf{a}_{t,i} | \mathbf{s}_{t,i}) \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i})$

5. Update $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



“actor-critic algorithm”

$$y_{i,t} = \sum_{t'=t}^{t+n-1} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{i,t+n})$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$